

TagScan

Raspberry Pi to VPS

CSV delivery over HTTPS

The complete step-by-step guide for a first-timer:
from an empty server to CSV files arriving in TagScan by themselves.

Prepared 20 September 2026 | Module 1 (TagScan) | Environments: test.rwcrew.eu and rwcrew.eu

Contents

Part 0. Read this first	5
0.1 What we are building (in plain words)	5
0.2 Words you will meet (glossary)	5
0.3 The three places you will type commands	6
0.4 The plan: 10 parts, test first, production last	6
0.5 What you need before you start	7
Part 1. Save the code changes on your PC	8
Step 1.1 Check the tests still pass (optional but reassuring)	8
Step 1.2 Commit and push to the develop branch	8
Part 2. Find out where the server stands	9
Part 3. Prepare the VPS and get HTTPS working	10
Step 3.1 Create the VPS	10
Step 3.2 Point the domain names at the VPS (DNS)	10
Step 3.3 First login to the VPS	10
Step 3.4 Create a safe everyday user and lock the door	11
Step 3.5 Turn on the firewall	12
Step 3.6 Install updates (and automatic security updates)	13
Step 3.7 Install Docker	13
Step 3.8 Install nginx and get the HTTPS certificate	14
Part 4. Install RWCrew on the VPS (test site)	15
Step 4.1 Download the code	15
Step 4.2 Create the secrets file (.env)	15
Step 4.3 Build and start everything	16
Step 4.4 Create the database tables and your first admin	16
Step 4.5 Check the site	16
Part 5. Optional: automatic deploys from GitHub	18
Step 5.1 Create a key just for GitHub	18
Step 5.2 Allow that key on the VPS	18
Step 5.3 Give GitHub the three secrets	18
Step 5.4 Try it	18
Part 6. Create the scanner and its API key in TagScan	19
Step 6.1 Give yourself access to TagScan	19
Step 6.2 Register the Raspberry Pi as a scanner	19

Step 6.3 Generate the API key	19
Step 6.4 Copy the upload address	19
Part 7. Prove the chain from your PC first	20
Step 7.1 Make a small test CSV	20
Step 7.2 Upload it	20
Step 7.3 Prove the safety features	20
Step 7.4 The size test (proves the nginx fix)	20
Step 7.5 See it in TagScan	21
If something failed	21
Part 8. The Raspberry Pi	22
Step 8.1 Log in to the Pi	22
Step 8.2 Check the Pi can reach the VPS	22
Step 8.3 Copy the watcher files from your PC to the Pi	22
Step 8.4 Install the files (on the Pi)	23
Step 8.5 Create the folders the files travel through	23
Step 8.6 Write the configuration file	23
Step 8.7 Start the watcher	24
Step 8.8 Send a test file from the Pi	24
Step 8.9 See it arrive	24
Step 8.10 Connect your real Pi application	25
Part 9. Repeat everything for production (rwcrew.eu)	26
Step 9.1 Bring the new code to the main branch (on your PC)	26
Step 9.2 Install the production copy on the VPS	26
Step 9.3 Access, scanner and key on the production site	27
Step 9.4 Add the production watcher on the Pi	27
Step 9.5 Switch your application over to production	27
Part 10. Daily use and looking after the system	28
How you use it every day	28
How to see that everything is healthy	28
Housekeeping	28
Troubleshooting table	29
Appendix A. Command cheat sheet	30
Appendix B. Worksheet: values to write down	31
Appendix C. Final checklist	32

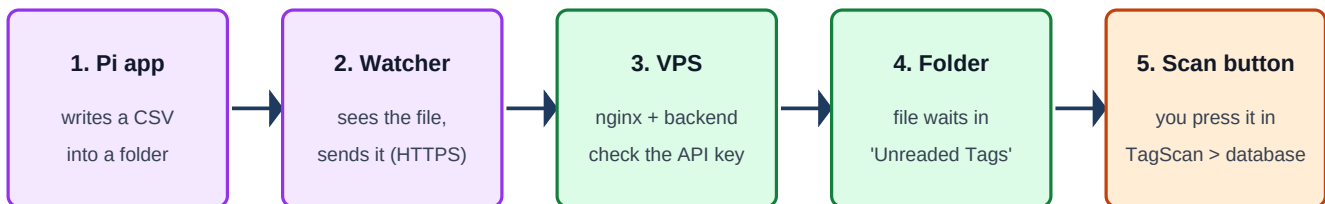
Appendix D. The CSV file format TagScan expects

33

Part 0. Read this first

0.1 What we are building (in plain words)

Your Raspberry Pi already runs an application that creates CSV files (lists of scanned RFID tags) and puts them in a folder. You want those files to end up on your server on the internet (the **VPS**) so the **TagScan** module of RWCrew can use them. This guide sets that up so it happens **automatically**, every few seconds, without anyone copying files by hand.



Steps 1-2 happen on the Raspberry Pi.

Steps 3-4 happen on the VPS (your server on the internet), automatically.

Step 5 is a human clicking 'Scan' in the TagScan web pages (it stays manual, by your choice).

Most of the software for this **already exists** in your project: the upload address on the server, the secret keys that identify each scanner, and a small program for the Pi called the **watcher**. What is missing is that nothing is switched on yet: the server does not have HTTPS (the padlock), the watcher is not installed on the Pi, and no key has been generated. This guide does all of that, in the right order.

0.2 Words you will meet (glossary)

Word	What it means
VPS	A rented computer in a data centre that is always on (yours is at Hostinger). It runs the RWCrew website and database. You control it by typing commands over the internet.
SSH	The way you 'log in' to the VPS or the Pi from your PC and type commands there. You will open SSH windows to two different machines - keep track of which is which!
DNS	The internet's phone book. A DNS <i>A record</i> says 'rwcrow.eu means the server with this IP address'.
HTTPS / certificate	The padlock in the browser. The connection is encrypted, so nobody on the way can read your secret key. It needs a <i>certificate</i> , which we get for free from Let's Encrypt using a tool called certbot .
nginx	A program on the VPS that receives all internet traffic first and hands it to the right part of RWCrew. It is also where the maximum upload size and the speed limit are set.
Docker	A way of running RWCrew's parts (website, backend, database) in separate 'containers', so they do not disturb the server itself.
Git / branch	Your code history. develop is the branch that becomes the test site; main becomes the real (production) site.
API key	A long secret text that proves 'this upload really comes from scanner X'. Treat it like a password.
Watcher	A small program on the Pi that looks in a folder every 15 seconds, sends new CSV files to the VPS and moves the sent ones away.
systemd service	The Linux way of keeping a program running in the background, starting it at boot and restarting it if it crashes.

Word	What it means
.env file	A small text file with secrets and settings (passwords, addresses). It is never put in Git.

0.3 The three places you will type commands

Every command box in this guide has a **coloured strip on top** telling you where to run it. Mixing these up is the number one beginner mistake, so please always check the strip before you paste anything.

Strip colour	Where	How you get there
BLUE	Your Windows PC	Start menu > type 'PowerShell' > open Windows PowerShell . The prompt looks like PS C:\Users\frede>
GREEN	The VPS (server)	From PowerShell: ssh deploy@YOUR_VPS_IP (explained in Part 3). The prompt looks like deploy@something:~\$
PURPLE	The Raspberry Pi	From PowerShell: ssh YOUR_PI_USER@YOUR_PI_HOST (explained in Part 8). The prompt looks like pi@raspberrypi:~\$
ORANGE	Your web browser	Chrome, Edge, Firefox... no commands, just clicking.

NOTE

When a command has a word in capitals such as **YOUR_VPS_IP** or **YOUR_API_KEY**, that is a **placeholder**: replace the whole word with your real value. Appendix B is a worksheet where you write all of these values down as you collect them.

WATCH OUT

Copying commands out of a PDF is sometimes flaky (line breaks, odd quote characters). If a pasted command gives a strange error, type it by hand instead. Every command in this guide is kept short enough to fit on one line of the box. Blocks that use a trailing `\` (server) or a trailing backtick (PowerShell) are **one** command spread over several lines; paste the whole block.

NOTE

Typing a password in SSH shows nothing - no dots, no stars. That is normal. Just type it and press Enter. To paste into an SSH window, **right-click** (Windows PowerShell) instead of Ctrl+V.

0.4 The plan: 10 parts, test first, production last

RWCrew has two copies: a **test** site (test.rwcrew.eu) and the real **production** site (rwcrew.eu). You will build and prove everything on **test** first, so a mistake cannot hurt real data. Only when it works do you repeat it for production (Part 9).

Part	What you do	Where	Time
1	Save the code changes made in the repository (Git commit and push)	PC	10 min
2	Find out how far the server is already set up	PC + VPS	10 min
3	Prepare the VPS: DNS, safe login, firewall, Docker, nginx, HTTPS	PC + VPS	60-90 min
4	Install RWCrew on the VPS (test site) and log in	VPS + browser	45 min
5	<i>Optional</i> : automatic deploys from GitHub	PC + VPS + GitHub	30 min
6	In TagScan: create the scanner, generate the API key, copy the upload address	Browser	15 min
7	Prove the whole chain from your PC before touching the Pi	PC + browser	15 min
8	Install the watcher on the Raspberry Pi and connect your Pi application	Pi	60 min
9	Repeat for production (rwcrew.eu)	All	60-90 min

Part	What you do	Where	Time
10	Daily use, housekeeping, backups	-	-

Plan for two sessions: Parts 1-4 (server) in one, Parts 5-9 in another. You can stop safely at the end of any part.

0.5 What you need before you start

- [] A Hostinger **VPS** (any current plan; 2 CPU / 4 GB RAM is comfortable) - or the login to buy/create one.
- [] Control of the **DNS** of **rwcrew.eu** (in Hostinger's hPanel if the domain is there, otherwise at your registrar).
- [] Admin access to the GitHub repository **FreewoodRWCrew/RWCrew**.
- [] The Windows PC where the RWCrew repository lives (C:\Users\frede\Documents\GitHub\RWCrew) with Git installed.
- [] The Raspberry Pi: powered on, on the network, with its CSV-creating application working, and you know its login (user + password) and its address (name like tagscan-pi-01.local, or an IP).
- [] About 1-2 hours of undisturbed time per session, and a password manager (or notebook) to keep secrets.

IMPORTANT

Never paste your API keys, passwords or .env files into chat, email or Git. The only places they belong are: the password manager, the .env file on the VPS, and the env file on the Pi.

Part 1. Save the code changes on your PC

While preparing this guide, four things were changed in the repository. They are needed for the upload to work well, so they must be committed and pushed to GitHub before the server can use them:

File	What changed
deploy/nginx/rwcrew.conf	Allows uploads up to 25 MB (nginx's default of 1 MB would reject bigger CSV files) and limits how fast one address may hit the upload URL (protection against password guessing).
backend/.../device_router.py	If a file arrives with a name that exists but different content , it is now stored under a new name instead of being thrown away. An identical re-send is still recognised as a harmless duplicate.
backend/tests/.../test_device_intake.py	Automatic tests for the above (13 tests, all passing).
CLAUDE.md	Short project notes about this feature.

Step 1.1 Check the tests still pass (optional but reassuring)

RUN ON: YOUR WINDOWS PC (PowerShell)

```
cd C:\Users\frede\Documents\GitHub\RWCrew\backend
.venv\Scripts\python -m pytest tests/modules/module_1/test_device_intake.py -q
```

YOU KNOW IT WORKED WHEN

The last line says **13 passed**.

Step 1.2 Commit and push to the develop branch

The backtick at the end of a line tells PowerShell 'the command continues on the next line'. Paste the whole block.

RUN ON: YOUR WINDOWS PC (PowerShell)

```
cd C:\Users\frede\Documents\GitHub\RWCrew
git status
git checkout develop
git add deploy/nginx/rwcrew.conf CLAUDE.md `
    backend/app/modules/module_1/device_router.py `
    backend/tests/modules/module_1/test_device_intake.py `
    docs/tagscan-raspi-to-vps-beginner-guide.pdf
git commit -m "TagScan intake: upload limit, rate limit, safe duplicate handling"
git push -u origin develop
```

NOTE

Git may ask you to log in to GitHub the first time (a browser window opens). Pushing to **develop** only updates the *test* copy - and only once automatic deploys exist (Part 5). Production is untouched until Part 9.

YOU KNOW IT WORKED WHEN

git push ends with a line like 'develop -> develop', and the branch is visible on github.com/FrewoodRWCrew/RWCrew.

Part 2. Find out where the server stands

You told me HTTPS is not live yet, but I cannot see your server. Use this short quiz to find your starting point. Work down the list; the first 'No' is where you continue in Part 3. Everything above it you can skip.

Question	How to check	If NO, go to
Do you have a VPS with Ubuntu 24.04 and know its IP address?	Hostinger hPanel > VPS. The IP address is on the overview page.	Step 3.1
Do the DNS names point to it?	PowerShell: <code>nslookup test.rwcrew.eu</code> shows your VPS IP as the answer.	Step 3.2
Can you log in as the normal user deploy with a key?	PowerShell: <code>ssh deploy@YOUR_VPS_IP</code> logs in without asking for a password.	Steps 3.3 - 3.4
Is the firewall on?	On the VPS: <code>sudo ufw status</code> shows 'Status: active'.	Step 3.5
Is Docker installed?	On the VPS: <code>docker compose version</code> prints a version.	Step 3.7
Is nginx installed with the certificate?	Browser: <code>https://test.rwcrew.eu</code> shows a padlock (even a '502 Bad Gateway' page is fine).	Step 3.8
Is RWCrew running on the VPS?	On the VPS: <code>ls /opt/rwcrew</code> lists a folder; the site loads and you can log in.	Part 4

NOTE

If a step turns out to be partly done already, do not redo it blindly. Run its 'you know it worked when' check; if that passes, move on.

Part 3. Prepare the VPS and get HTTPS working

This part follows your existing Hostinger guide (docs/deploying-to-hostinger.pdf), with Windows-friendly commands and the two nginx changes built in.

Step 3.1 Create the VPS

1. Log in to **Hostinger hPanel > VPS > your VPS > Operating System**.
2. Choose **Ubuntu 24.04 LTS**. If Hostinger offers 'Ubuntu 24.04 with Docker', pick that: it lets you skip Step 3.7.
3. Set a strong **root password** and write it in your password manager.
4. When installation finishes, note the VPS **public IPv4 address** on the overview page. This guide calls it **YOUR_VPS_IP**. Write it in Appendix B.

Step 3.2 Point the domain names at the VPS (DNS)

In the DNS editor of rwcrow.eu (hPanel > Domains > rwcrow.eu > DNS / Nameservers > DNS Zone Editor) create three **A records**. Remove any old A records with the same names first.

Type	Name	Points to	TTL
A	@	YOUR_VPS_IP	3600 (or default)
A	www	YOUR_VPS_IP	3600
A	test	YOUR_VPS_IP	3600

Wait a few minutes, then check from your PC:

RUN ON: YOUR WINDOWS PC (PowerShell)

```
nslookup rwcrow.eu
nslookup test.rwcrow.eu
```

YOU KNOW IT WORKED WHEN

Both answers show your VPS IP address. If they show a different address or 'Non-existent domain', wait 10-30 minutes and try again (DNS can be slow). Do not continue to certbot before this works.

Step 3.3 First login to the VPS

RUN ON: YOUR WINDOWS PC (PowerShell)

```
ssh root@YOUR_VPS_IP
```

The first time, Windows asks '*Are you sure you want to continue connecting?*'. Type **yes** and press Enter, then type the root password (nothing is shown while typing).

YOU KNOW IT WORKED WHEN

The prompt changes to **root@something:~#**. You are now typing on the server. Everything in the next two steps is run there, as root.

Step 3.4 Create a safe everyday user and lock the door

Working as root all the time is dangerous. You will create a user called **deploy**, give it your PC's SSH key, and then forbid password logins.

a) Create the user (on the VPS, as root)

RUN ON: THE VPS (inside your SSH session)

```
adduser deploy
usermod -s /usr/sbin/sudo_sudo deploy
```

adduser asks for a password (choose one and save it) and some optional details (just press Enter for each).

b) Get your PC's public SSH key (on your PC)

If the first command prints **True**, you already have a key. If it prints **False**, run the second command and press Enter at every question.

RUN ON: YOUR WINDOWS PC (PowerShell)

```
Test-Path $env:USERPROFILE\.ssh\id_ed25519.pub
ssh-keygen -t ed25519
Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub
```

The last command prints one long line starting with **ssh-ed25519**. Select it fully and copy it (this is the *public* key; it is safe to share, unlike a private key).

c) Install that key for user deploy (on the VPS, as root)

RUN ON: THE VPS (inside your SSH session)

```
mkdir -p /home/deploy/.ssh
nano /home/deploy/.ssh/authorized_keys
```

How to use nano

nano is a simple text editor in the terminal. Right-click to paste your key line, then press **Ctrl+O**, **Enter** (save), and **Ctrl+X** (leave). The mouse does not work inside nano; use the arrow keys.

RUN ON: THE VPS (inside your SSH session)

```
chown -R deploy:deploy /home/deploy/.ssh
chmod 700 /home/deploy/.ssh
chmod 600 /home/deploy/.ssh/authorized_keys
```

d) Test the new login in a SECOND window (very important)

Open a **new** PowerShell window (leave the root window open!) and run:

RUN ON: YOUR WINDOWS PC (PowerShell)

```
ssh deploy@YOUR_VPS_IP
sudo whoami
```

YOU KNOW IT WORKED WHEN

You get in without typing the server password (a key passphrase prompt is fine), and **sudo whoami** prints **root**.

IMPORTANT

Do **not** continue with (e) until (d) works. Otherwise you can lock yourself out of your own server.

e) Forbid root and password logins (in the deploy session, using sudo)

RUN ON: THE VPS (inside your SSH session)

```
sudo nano /etc/ssh/sshd_config
```

Find these two lines (use the arrow keys; they may start with a # which you must remove) and make them read exactly:

FILE CONTENT

```
PermitRootLogin no
PasswordAuthentication no
```

Save (Ctrl+O, Enter, Ctrl+X). Ubuntu can override this setting in a second place, so check it and then restart the SSH service:

RUN ON: THE VPS (inside your SSH session)

```
sudo grep -rn "PasswordAuthentication" /etc/ssh/sshd_config /etc/ssh/sshd_config.d/
sudo systemctl restart ssh
```

NOTE

If the grep output shows a line **PasswordAuthentication yes** inside a file in sshd_config.d (often 50-cloud-init.conf), edit that file the same way (**sudo nano** on that file, change yes to no), then restart ssh again. Finally, confirm in a third window that **ssh deploy@YOUR_VPS_IP** still works.

From now on you always log in with **ssh deploy@YOUR_VPS_IP** and use **sudo** for administrator commands. You can close the root window.

Step 3.5 Turn on the firewall

Order matters: allow SSH first, or you cut off your own connection.

RUN ON: THE VPS (inside your SSH session)

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status
```

Answer **y** to the warning about SSH connections (you allowed OpenSSH, so it is safe).

YOU KNOW IT WORKED WHEN

ufw status says 'Status: active' and lists OpenSSH, 80/tcp and 443/tcp.

NOTE

Hostinger's hPanel also has its own firewall feature. If you ever enabled it, make sure ports 22, 80 and 443 are allowed there too.

Step 3.6 Install updates (and automatic security updates)

RUN ON: THE VPS (inside your SSH session)

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y unattended-upgrades
```

If a purple screen asks about restarting services, press Enter to accept the defaults. If it says a reboot is needed, run `sudo reboot`, wait a minute, and log in again.

Step 3.7 Install Docker

Skip this if you chose Hostinger's 'Ubuntu with Docker' template (check with `docker --version`). Otherwise run these blocks one after the other:

RUN ON: THE VPS (inside your SSH session)

```
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

RUN ON: THE VPS (inside your SSH session)

```
echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

RUN ON: THE VPS (inside your SSH session)

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker deploy
```

Now **log out and back in** so the new group membership is active:

RUN ON: THE VPS (inside your SSH session)

```
exit
```

RUN ON: YOUR WINDOWS PC (PowerShell)

```
ssh deploy@YOUR_VPS_IP
docker --version
docker compose version
docker run --rm hello-world
```

YOU KNOW IT WORKED WHEN

Versions are printed, and hello-world says 'Hello from Docker!'.

Step 3.8 Install nginx and get the HTTPS certificate

a) Install the programs (on the VPS)

RUN ON: THE VPS (inside your SSH session)

```
sudo apt install -y nginx certbot python3-certbot-nginx
```

b) Copy the nginx configuration from your PC to the VPS

Use **your local copy** of the file (it contains today's upload-limit changes). Do not download it from GitHub's main branch: that copy does not have the changes yet. Run this on your PC, in the repository folder (the VPS file goes to a temporary place first):

RUN ON: YOUR WINDOWS PC (PowerShell)

```
cd C:\Users\frede\Documents\GitHub\RWCrew
scp deploy\nginx\rwcrew.conf deploy@YOUR_VPS_IP:/tmp/rwcrew.conf
```

Then on the VPS, put it in place, activate it and check it for typing errors:

RUN ON: THE VPS (inside your SSH session)

```
sudo mv /tmp/rwcrew.conf /etc/nginx/sites-available/rwcrew.conf
sudo ln -sf /etc/nginx/sites-available/rwcrew.conf /etc/nginx/sites-enabled/rwcrew.conf
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

YOU KNOW IT WORKED WHEN

nginx -t prints 'syntax is ok' and 'test is successful'.

c) Ask for the certificates (needs the DNS from Step 3.2)

RUN ON: THE VPS (inside your SSH session)

```
sudo certbot --nginx -d rwcrew.eu -d www.rwcrew.eu -d test.rwcrew.eu
```

Certbot asks: your **email** (for expiry warnings), to agree to the terms (**Y**), and whether to share your email with the EFF (**N** is fine). If it asks whether to redirect HTTP to HTTPS, choose **redirect**. Certbot then edits the nginx file by itself to add the HTTPS parts.

d) Verify

RUN ON: THE VPS (inside your SSH session)

```
sudo grep -nE "client_max_body_size|limit_req|listen 443" \
/etc/nginx/sites-available/rwcrew.conf
sudo certbot renew --dry-run
```

YOU KNOW IT WORKED WHEN

The grep shows **client_max_body_size 25m** twice (once per site) and several **listen 443** lines; the dry run ends with 'Congratulations, all simulated renewals succeeded'. In your browser, **https://test.rwcrew.eu** shows a padlock. A '502 Bad Gateway' page is **expected** right now - RWCrew is not installed yet (that is Part 4).

WATCH OUT

Certbot fails? The usual causes: DNS not yet pointing to the VPS (repeat Step 3.2 checks), or port 80 blocked by a firewall (Step 3.5 / hPanel firewall). Fix that and run the certbot command again. It is safe to repeat. Let's Encrypt allows only a few failed attempts per hour, so do not spam it.

Part 4. Install RWCrew on the VPS (test site)

You are still on the VPS as user **deploy**. You will install the **test** copy first (branch **develop**, address <https://test.rwcrew.eu>).

Step 4.1 Download the code

RUN ON: THE VPS (inside your SSH session)

```
sudo mkdir -p /opt/rwcrew
sudo chown deploy:deploy /opt/rwcrew
git clone -b develop https://github.com/FreewoodRWCrew/RWCrew.git /opt/rwcrew/test
```

Private repository?

If Git asks for a username and password, the repository is **private**. Give the VPS a read-only key instead: (1) run the three lines below, (2) on GitHub open the repository > **Settings** > **Deploy keys** > **Add deploy key**, paste the printed key, leave 'Allow write access' **unticked**, (3) clone with the `git@github.com` address shown after the block.

RUN ON: THE VPS (inside your SSH session)

```
ssh-keygen -t ed25519 -C "vps-readonly" -f ~/.ssh/github_deploy
cat ~/.ssh/github_deploy.pub
printf 'Host github.com\n  IdentityFile ~/.ssh/github_deploy\n' >> ~/.ssh/config
printf '  IdentitiesOnly yes\n' >> ~/.ssh/config
```

RUN ON: THE VPS (inside your SSH session)

```
git clone -b develop git@github.com:FreewoodRWCrew/RWCrew.git /opt/rwcrew/test
```

YOU KNOW IT WORKED WHEN

The folder `/opt/rwcrew/test` exists and `ls /opt/rwcrew/test` shows files such as `docker-compose.prod.yml`.

Step 4.2 Create the secrets file (.env)

RUN ON: THE VPS (inside your SSH session)

```
cd /opt/rwcrew/test
cp .env.test.example .env
python3 -c "import secrets; print(secrets.token_hex(32))"
python3 -c "import secrets; print(secrets.token_hex(32))"
```

The two **python3** commands each print a long random text. Copy them one at a time into a notebook: the **first** is your database password, the **second** is your JWT secret (used to sign logins). Now edit the file:

RUN ON: THE VPS (inside your SSH session)

```
nano .env
```

Change exactly three places. The database password appears **twice** (it must be identical). Example, with **FIRST** and **SECOND** standing for your two random values:

FILE CONTENT

```
POSTGRES_PASSWORD=FIRST
DATABASE_URL=postgres://rwcrew:FIRST@db:5432/rwcrew
JWT_SECRET=SECOND
```

Leave everything else (addresses, ports, TEST label) as it is. Save with **Ctrl+O**, **Enter**, **Ctrl+X**, then protect the file:

RUN ON: THE VPS (inside your SSH session)

```
chmod 600 .env
```

IMPORTANT

The test and the production .env files must use **different** random values. Never commit a .env file to Git (it is already ignored).

Step 4.3 Build and start everything

RUN ON: THE VPS (inside your SSH session)

```
docker compose -f docker-compose.prod.yml up -d --build
```

This downloads and builds everything and takes **5-10 minutes** the first time; lots of text scrolls by. When your prompt returns, check the containers:

RUN ON: THE VPS (inside your SSH session)

```
docker compose -f docker-compose.prod.yml ps
```

YOU KNOW IT WORKED WHEN

Three services (db, backend, frontend) show **running** (or 'Up').

NOTE

Something not running? Read its log: `docker compose -f docker-compose.prod.yml logs backend` (or frontend / db). The most common cause is a typo in .env - fix it, then run the 'up -d --build' command again.

Step 4.4 Create the database tables and your first admin

RUN ON: THE VPS (inside your SSH session)

```
docker compose -f docker-compose.prod.yml exec backend \
python -m alembic upgrade head
docker compose -f docker-compose.prod.yml exec backend \
python -m app.cli.seed --email you@example.com \
--password 'PICK-A-STRONG-PASSWORD' --name 'Your Name'
```

Use your real email. Keep the **single quotes** around the password. Avoid exclamation marks in it (the shell treats them specially).

YOU KNOW IT WORKED WHEN

The first command lists migration steps without errors; the second prints that the super admin was created.

Step 4.5 Check the site

RUN ON: THE VPS (inside your SSH session)

```
curl -i https://test.rwcrew.eu/api/health
```

YOU KNOW IT WORKED WHEN

The first line says **HTTP/2 200**.

Now in your browser open **https://test.rwcrew.eu** and log in with the email and password from Step 4.4. A banner saying **TEST** is shown on every page; that is intended.

WATCH OUT

Change the admin password after your first login, since it was typed on the command line (it is stored in the shell history).

Part 5. Optional: automatic deploys from GitHub

With this, pushing to **develop** updates the test site by itself (and pushing to **main** updates production). It is optional: without it you update by running one command on the VPS (see the note at the end). The workflows already exist in the repository (.github/workflows); they only need credentials.

Step 5.1 Create a key just for GitHub

On your PC. Press Enter at both passphrase questions (empty passphrase, because a robot uses this key):

RUN ON: YOUR WINDOWS PC (PowerShell)

```
cd $env:USERPROFILE
ssh-keygen -t ed25519 -C "github-actions-deploy" -f gh_deploy_key
Get-Content gh_deploy_key.pub
```

Step 5.2 Allow that key on the VPS

Copy the line that was just printed. On the VPS (as deploy), add it as a **new line at the end** of the file (do not delete the existing line):

RUN ON: THE VPS (inside your SSH session)

```
nano ~/.ssh/authorized_keys
```

Step 5.3 Give GitHub the three secrets

On github.com: repository > **Settings** > **Secrets and variables** > **Actions** > **New repository secret**. Create:

Name	Value
SSH_HOST	YOUR_VPS_IP
SSH_USER	deploy
SSH_PRIVATE_KEY	The whole contents of the private key file, including the BEGIN and END lines. Show it with: Get-Content gh_deploy_key -Raw

Afterwards delete the key files from the PC (they are now stored in GitHub and on the VPS):

RUN ON: YOUR WINDOWS PC (PowerShell)

```
Remove-Item $env:USERPROFILE\gh_deploy_key, $env:USERPROFILE\gh_deploy_key.pub
```

Step 5.4 Try it

Make a tiny change on develop (for example add a blank line to a README), commit, push, and open the repository's **Actions** tab on GitHub.

YOU KNOW IT WORKED WHEN

A run called 'Deploy test' finishes with a green tick within a few minutes.

NOTE

Without automatic deploys, update the test site any time from the VPS with: **bash /opt/rwcrew/test/deploy/scripts/deploy.sh /opt/rwcrew/test**. It pulls the newest code, rebuilds, restarts and applies database changes.

Part 6. Create the scanner and its API key in TagScan

This is done in the **web app**, on the **test** site. You need two things from it: an **API key** and the **upload address**.

Step 6.1 Give yourself access to TagScan

RWCrew keeps 'who may open which module' separate from being super admin: even a super admin does not see a module until access is granted. Log in to <https://test.rwcrew.eu>, open the **Manage Access** screen of the admin area, and tick the **TagScan** box on your own user's row. Go back to the start page: the TagScan tile now appears.

Step 6.2 Register the Raspberry Pi as a scanner

1. Open **Modules > TagScan**. In the left menu, under **MasterData**, open **Scanners**.
2. Click **New scanner**. Fill in: **Name** (for example **Scan-01**; use the same name your Pi application writes in the 'Scanner' column of its CSV files), **Type**, and **Technology** (for example Raspberry Pi 4).
3. If the **Type** list is empty, first add one on the product types screen in the same MasterData menu (button 'New type', for example 'Reader Hardware'), then come back.
4. Save.

Step 6.3 Generate the API key

1. In the Scanners table, find the row of your scanner and click the **key icon** in its Actions column.
2. Click **Generate API key**.
3. The key is shown **only this once**. Click **Copy** and immediately store it in your password manager and in Appendix B. It looks like `module1_2_AbC...long...`

IMPORTANT

The key cannot be shown again. If you lose it, click **Generate new key**: that makes a new one and instantly **invalidates the old one**.

Step 6.4 Copy the upload address

In the TagScan left menu open **Settings** and copy the **Device upload URL** (use its Copy button). For the test site it must read:

FILE CONTENT

```
https://test.rwcrew.eu/api/public/tagscan-intake
```

WATCH OUT

It must start with **https://** and contain **test.rwcrew.eu**. If it shows localhost or a different host name, the `NEXT_PUBLIC_API_URL` line in `/opt/rwcrew/test/.env` is wrong: fix it and rebuild with `docker compose -f docker-compose.prod.yml up -d --build`.

YOU KNOW IT WORKED WHEN

You now hold two values: **YOUR_API_KEY** and **YOUR_UPLOAD_URL**.

Part 7. Prove the chain from your PC first

Before touching the Pi, upload a file from your Windows PC. If this works, the server side is finished, and any later problem must be on the Pi. This saves hours of guessing.

Step 7.1 Make a small test CSV

The column names must be exactly those below (this is what TagScan reads). Run in PowerShell:

RUN ON: YOUR WINDOWS PC (PowerShell)

```
cd $env:USERPROFILE\Desktop
Set-Content -Path test-scan.csv -Encoding ascii -Value `
  "Scanner,EPC,RSSI (raw),Antenna,Count,Last Seen", `
  "Scan-01,E2801160600002054F6A0001,-55,1,3,2026-09-20 10:15:00"
```

Step 7.2 Upload it

Type **curl.exe** (with .exe): plain 'curl' in Windows PowerShell is a different, unrelated command.

RUN ON: YOUR WINDOWS PC (PowerShell)

```
curl.exe -i -H "X-API-Key: YOUR_API_KEY" `
  -F "file=@test-scan.csv" `
  https://test.rwcrew.eu/api/public/tagscan-intake
```

YOU KNOW IT WORKED WHEN

The reply contains **HTTP/1.1 201** or **HTTP/2 201** and **{"status":"received","filename":"test-scan.csv"}**.

Step 7.3 Prove the safety features

Try this	You should see	It proves
Run the same curl command again	201 with status duplicate	Re-sending is harmless (the Pi can safely retry)
Change one letter of the key and run it	401 Invalid API key	Wrong keys are rejected
Rename the file to test-scan.txt and upload it	400 Only .csv files are accepted	Only CSVs are accepted
Upload a 4 MB file (Step 7.4)	201 , not 413	The nginx 25 MB limit is active

Step 7.4 The size test (proves the nginx fix)

RUN ON: YOUR WINDOWS PC (PowerShell)

```
$h = "Scanner,EPC,RSSI (raw),Antenna,Count,Last Seen"
$rows = 1..60000 | ForEach-Object {
  "Scan-01,E2801160600002054F6A{0:D8},-55,1,3,2026-09-20 10:15:00" -f $_ }
Set-Content -Path big-test.csv -Encoding ascii -Value (@($h) + $rows)
(Get-Item big-test.csv).Length
```

The last line prints the size in bytes (about 4 million). Upload it exactly like Step 7.2 but with **-F "file=@big-test.csv"**.

NOTE

Only for the test site: this file will sit in 'Unreaded Tags' too. Simply do not press Scan while it is there, or remove it on the VPS with: `docker compose -f docker-compose.prod.yml exec backend rm '/app/TagScans/Unreaded Tags/big-test.csv'` (run inside /opt/rwcrew/test).

Step 7.5 See it in TagScan

1. In the browser open **Modules > TagScan > CSV Source Files** (the Dashboard) and browse into the folder **Unreaded Tags**. Your test-scan.csv is listed.
2. Open **Tag Headerdata** and press **Scan**. The file is read, its lines are stored, and it moves to the **Read Tags** folder.
3. Open the Scanners screen again: the key of your scanner now shows **Last used** with a recent time.

YOU KNOW IT WORKED WHEN

All three things happened. The server side is done!

If something failed

What you see	Likely cause	What to do
Could not resolve host / SSL error	DNS or certificate not ready	Redo Step 3.2 checks and Step 3.8; open https://test.rwcrew.eu in a browser to check the padlock.
502 Bad Gateway	Backend container not running	On the VPS in /opt/rwcrew/test: <code>docker compose -f docker-compose.prod.yml ps</code> , then <code>logs backend</code> .
401 Invalid API key	Key typed wrong, or generated on the other environment	Copy it again; keys only work on the site where they were generated.
413 Request Entity Too Large (with nginx in the text)	nginx limit missing	Check Step 3.8d: <code>client_max_body_size 25m</code> must appear twice.
413 without nginx text	File larger than 20 MB	The app's own limit (setting <code>tagscan_intake_max_file_mb</code>).
404 Not Found	Wrong address	It must end exactly in <code>/api/public/tagscan-intake</code> .
503 Service Temporarily Unavailable	Speed limit hit (30 uploads/minute per address)	Wait a minute; the watcher retries automatically.

Part 8. The Raspberry Pi

Your Pi is already running and working with your application, so this part only **adds the watcher**. If the Pi is not set up at all yet, first follow sections 3-4 of docs/tagscan-raspi-setup-guide.pdf (flashing the SD card and first boot).

NOTE

Naming used below: the watcher can run several times on one Pi, each with a name. We call the instance for the test site **dev** and the one for production **prod** (the names used in the project's existing scripts). Right now you set up **dev** = test.rwcrew.eu.

Step 8.1 Log in to the Pi

RUN ON: YOUR WINDOWS PC (PowerShell)

```
ssh YOUR_PI_USER@YOUR_PI_HOST
```

For example `ssh pi@tagscan-pi-01.local`. If the name does not work, use the Pi's IP address (visible in your router's device list).

Step 8.2 Check the Pi can reach the VPS

RUN ON: THE RASPBERRY PI (inside your SSH session)

```
date
python3 --version
curl -i https://test.rwcrew.eu/api/health
```

YOU KNOW IT WORKED WHEN

date shows today's correct date and time (a Pi with a wrong clock cannot verify HTTPS certificates), Python 3 is present, and the curl reply starts with **HTTP/2 200**.

WATCH OUT

If the date is wrong, run `sudo timedatectl set-ntp true` and check again after a minute. Without a correct clock, the upload will fail with a certificate error.

Step 8.3 Copy the watcher files from your PC to the Pi

RUN ON: YOUR WINDOWS PC (PowerShell)

```
cd C:\Users\frede\Documents\GitHub\RWCrew\scripts\pi-watcher
scp watcher.py YOUR_PI_USER@YOUR_PI_HOST:/tmp/watcher.py
scp tagscan-watcher@.service YOUR_PI_USER@YOUR_PI_HOST:/tmp/tagscan-watcher@.service
```

Step 8.4 Install the files (on the Pi)

RUN ON: THE RASPBERRY PI (inside your SSH session)

```
sudo mkdir -p /opt/tagscan-watcher /etc/tagscan-watcher
sudo mv /tmp/watcher.py /opt/tagscan-watcher/watcher.py
sudo mv /tmp/tagscan-watcher@.service /etc/systemd/system/tagscan-watcher@.service
sudo useradd --system --no-create-home --shell /usr/sbin/nologin tagscan-watcher
```

The last line creates a dedicated user without rights or a login: the watcher runs as this user, so even if it were abused it can do almost nothing.

Step 8.5 Create the folders the files travel through

The folder **outgoing** is where your Pi application drops finished CSV files. **sent** is where the watcher stores files after a successful upload. They are placed in /var/lib (not in /home) so that the watcher user is allowed to reach them.

RUN ON: THE RASPBERRY PI (inside your SSH session)

```
sudo mkdir -p /var/lib/tagscan/dev/outgoing /var/lib/tagscan/dev/sent
sudo chown -R tagscan-watcher:tagscan-watcher /var/lib/tagscan
sudo chmod -R 2775 /var/lib/tagscan
```

Your Pi application (running as its own user) must be allowed to write into **outgoing**. Find out which user it runs as, then add that user to the watcher's group:

RUN ON: THE RASPBERRY PI (inside your SSH session)

```
ps -eo user,args | grep -i YOUR_APP_NAME | grep -v grep
sudo usermod -aG tagscan-watcher YOUR_APP_USER
```

NOTE

YOUR_APP_NAME is any word in the command of your application (for example its script name). If you start the application by hand from your login, the user is simply **pi** (or your Pi login name). The group change becomes active after the application is restarted (or the Pi rebooted).

Step 8.6 Write the configuration file

RUN ON: THE RASPBERRY PI (inside your SSH session)

```
sudo nano /etc/tagscan-watcher/dev.env
```

Type these five lines, replacing the API key with your own from Step 6.3 (the URL is the one from Step 6.4). No quotes, no spaces around the equals sign:

FILE CONTENT

```
API_URL=https://test.rwcrew.eu/api/public/tagscan-intake
API_KEY=YOUR_API_KEY
WATCH_DIR=/var/lib/tagscan/dev/outgoing
SENT_DIR=/var/lib/tagscan/dev/sent
POLL_INTERVAL_SECONDS=15
```

Save (Ctrl+O, Enter, Ctrl+X) and lock the file, because it contains a secret:

RUN ON: THE RASPBERRY PI (inside your SSH session)

```
sudo chown root:root /etc/tagscan-watcher/dev.env
sudo chmod 600 /etc/tagscan-watcher/dev.env
```

Step 8.7 Start the watcher

RUN ON: THE RASPBERRY PI (inside your SSH session)

```
sudo systemctl daemon-reload
sudo systemctl enable --now tagscan-watcher@dev
sudo systemctl status tagscan-watcher@dev
```

The text after the @ in the name (**dev**) selects which configuration file it reads (dev.env). Press **q** to leave the status screen. Then watch the live log:

RUN ON: THE RASPBERRY PI (inside your SSH session)

```
sudo journalctl -u tagscan-watcher@dev -f
```

YOU KNOW IT WORKED WHEN

The status shows **active (running)** in green, and the log's first line reads: *Watching /var/lib/tagscan/dev/outgoing, sending to https://test.rwcrew.eu/api/public/tagscan-intake every 15.0s*. Press Ctrl+C to stop following the log (the service keeps running).

Step 8.8 Send a test file from the Pi

Leave the log running in this window. Open a **second** PowerShell window, log in to the Pi again (Step 8.1) and run the following. It creates the file under a **.tmp** name first and renames it when finished, exactly the way your real application must do it (Step 8.10):

RUN ON: THE RASPBERRY PI (inside your SSH session)

```
sudo -u tagscan-watcher tee /var/lib/tagscan/dev/outgoing/pi-test-001.csv.tmp \
> /dev/null <<'EOF'
Scanner,EPC,RSSI (raw),Antenna,Count,Last Seen
Scan-01,E2801160600002054F6A0002,-60,1,2,2026-09-20 10:20:00
EOF
sudo -u tagscan-watcher mv /var/lib/tagscan/dev/outgoing/pi-test-001.csv.tmp \
/var/lib/tagscan/dev/outgoing/pi-test-001.csv
```

Within **15 to 45 seconds** (the watcher waits until the file size has stopped changing for one round) the log window shows:

FILE CONTENT

```
Uploaded pi-test-001.csv -> HTTP 201
Moved pi-test-001.csv -> /var/lib/tagscan/dev/sent/pi-test-001.csv
```

Step 8.9 See it arrive

In the browser: TagScan > **CSV Source Files** > **Unreaded Tags** now lists **pi-test-001.csv**. Press **Scan** on the Tag Headerdata screen to load it into the database.

YOU KNOW IT WORKED WHEN

The file left the 'outgoing' folder, appeared in 'sent' on the Pi, and appeared in 'Unreaded Tags' on the server. The pipeline works from end to end.

Step 8.10 Connect your real Pi application

Now change your own application (you are able to modify it) so that it feeds the watcher. Three rules:

1. **Write into the outgoing folder:** `/var/lib/tagscan/dev/outgoing` (for now; production later gets its own folder).
2. **Write under a temporary name, then rename.** Write to **name.csv.tmp**, and only when the file is completely finished, rename it to **name.csv**. The watcher ignores anything not ending in `.csv`, so it never sends half-finished files. (A rename within one folder is instant and safe.)
3. **Use a unique file name every time**, never reused, for example **Scan-01_20260920T101500Z.csv** (scanner name + date and time in UTC). Reusing a name is now handled safely by the server, but unique names keep everything simple and traceable.

Example in Python (adapt to the language your application uses; the idea is the same everywhere):

FILE CONTENT

```
import csv, os
from datetime import datetime, timezone

OUT = "/var/lib/tagscan/dev/outgoing"
HEADER = ["Scanner", "EPC", "RSSI (raw)", "Antenna", "Count", "Last Seen"]

def save_scan(scanner, rows):
    stamp = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")
    final = os.path.join(OUT, f"{scanner}_{stamp}.csv")
    tmp = final + ".tmp" # ends in .tmp: watcher ignores it
    with open(tmp, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(HEADER)
        w.writerows(rows)
        f.flush()
        os.fsync(f.fileno()) # make sure the data is on disk
    os.replace(tmp, final) # instant rename: now it is ready
```

NOTE

Existing older CSV files on the Pi can be sent too: copy them into the outgoing folder (as `.csv` files that are complete). If you drop hundreds at once, some uploads may briefly get '503' because of the speed limit; the watcher simply retries them on the next round, nothing is lost.

WATCH OUT

CSV columns must be exactly: Scanner, EPC, RSSI (raw), Antenna, Count, Last Seen (see Appendix D). A file with other column names is uploaded fine but yields empty rows when you press Scan.

Part 9. Repeat everything for production (rwcrew.eu)

Only start this when Part 7 and Part 8 worked on the test site. The steps are the same; only the names, ports and keys differ. The Hostinger guide says: **main** branch = production, **develop** branch = test.

Step 9.1 Bring the new code to the main branch (on your PC)

RUN ON: YOUR WINDOWS PC (PowerShell)

```
cd C:\Users\frede\Documents\GitHub\RW Crew
git checkout main
git pull
git merge develop
git push
git checkout develop
```

NOTE

If you see a merge conflict message, stop and ask for help before continuing; do not guess.

Step 9.2 Install the production copy on the VPS

RUN ON: THE VPS (inside your SSH session)

```
git clone -b main https://github.com/FreewoodRW Crew/RWCrew.git /opt/rwcrew/production
cd /opt/rwcrew/production
cp .env.production.example .env
python3 -c "import secrets; print(secrets.token_hex(32))"
python3 -c "import secrets; print(secrets.token_hex(32))"
nano .env
```

Edit the .env exactly like Step 4.2, with **two new random values** (never the same as test). Leave the production addresses and ports (3000 / 8020) as shipped. If the repository is private, use the git@github.com address from Step 4.1. Then:

RUN ON: THE VPS (inside your SSH session)

```
chmod 600 .env
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml exec backend \
  python -m alembic upgrade head
docker compose -f docker-compose.prod.yml exec backend \
  python -m app.cli.seed --email you@example.com \
  --password 'PICK-A-STRONG-PASSWORD' --name 'Your Name'
curl -i https://rwcrew.eu/api/health
```

YOU KNOW IT WORKED WHEN

The health check returns 200, and https://rwcrew.eu shows the login page (without the TEST banner).

Step 9.3 Access, scanner and key on the production site

Repeat Steps 6.1 to 6.4 on <https://rwcrew.eu>. It has its own database, so you again need to grant yourself TagScan access, create the scanner, and generate a **new** API key. The upload URL becomes <https://rwcrew.eu/api/public/tagscan-intake>. Repeat Part 7 with this key and URL.

Step 9.4 Add the production watcher on the Pi

RUN ON: THE RASPBERRY PI (inside your SSH session)

```
sudo mkdir -p /var/lib/tagscan/prod/outgoing /var/lib/tagscan/prod/sent
sudo chown -R tagscan-watcher:tagscan-watcher /var/lib/tagscan
sudo chmod -R 2775 /var/lib/tagscan
sudo nano /etc/tagscan-watcher/prod.env
```

FILE CONTENT

```
API_URL=https://rwcrew.eu/api/public/tagscan-intake
API_KEY=YOUR_PRODUCTION_API_KEY
WATCH_DIR=/var/lib/tagscan/prod/outgoing
SENT_DIR=/var/lib/tagscan/prod/sent
POLL_INTERVAL_SECONDS=15
```

RUN ON: THE RASPBERRY PI (inside your SSH session)

```
sudo chown root:root /etc/tagscan-watcher/prod.env
sudo chmod 600 /etc/tagscan-watcher/prod.env
sudo systemctl enable --now tagscan-watcher@prod
sudo journalctl -u tagscan-watcher@prod -f
```

Step 9.5 Switch your application over to production

Change the output folder of your Pi application from `/var/lib/tagscan/dev/outgoing` to `/var/lib/tagscan/prod/outgoing`, restart it, and check that a new file reaches <https://rwcrew.eu> > TagScan > CSV Source Files > Unreaded Tags. When you are satisfied, switch off the test watcher so the two do not both hold files:

RUN ON: THE RASPBERRY PI (inside your SSH session)

```
sudo systemctl disable --now tagscan-watcher@dev
```

WATCH OUT

Never put the production key in dev.env or the test key in prod.env. A key only works on the site where it was generated (a wrong one gives 401), but mixing them up causes confusing evenings.

YOU KNOW IT WORKED WHEN

A CSV made by the real application appears in Unreaded Tags on rwcrew.eu within about a minute, and 'Last used' on the production Scanners screen updates.

Part 10. Daily use and looking after the system

How you use it every day

1. The Pi application creates CSV files; the watcher delivers them (within about 30 seconds each). Nothing to do.
2. In TagScan, open **Tag Headerdata** and press **Scan**. Every waiting file is read into the database and moved to Read Tags.
3. Use the Tag Headerdata and Tag Linedata screens to work with the results.

How to see that everything is healthy

Check	Where	Healthy looks like
Watcher running	Pi: <code>sudo systemctl status tagscan-watcher@prod</code>	active (running)
Recent uploads	Pi: <code>sudo journalctl -u tagscan-watcher@prod -n 50 --no-pager</code>	Lines 'Uploaded ... HTTP 201' with recent times
Nothing stuck on the Pi	Pi: <code>ls /var/lib/tagscan/prod/outgoing</code>	Empty, or only very recent files
Scanner talking to server	TagScan > Scanners > key icon	'Last used' is recent
Files waiting on server	TagScan > CSV Source Files > Unreaded Tags	Only files not yet scanned
Certificate renewing	VPS: <code>sudo certbot renew --dry-run</code>	'all simulated renewals succeeded'

If the internet is down the files simply stay in 'outgoing' and are uploaded when it returns. Nothing is lost and nothing is sent twice.

Housekeeping

- **Old files on the Pi.** The 'sent' folder keeps a copy of everything sent, so it grows. Once you trust the system, you can remove copies older than 30 days automatically. On the Pi run `sudo crontab -e` and add one line:

RUN ON: THE RASPBERRY PI (inside your SSH session)

```
30 3 * * * find /var/lib/tagscan/prod/sent -name '*.csv' -mtime +30 -delete
```

- **Updating the app.** Push to develop (test), check it, merge into main (production). With Part 5 this is automatic; otherwise run `deploy.sh` on the VPS (Step 5.4).
- **Backups.** Follow step 11 of the Hostinger guide (a nightly database backup) and also enable Hostinger's VPS snapshots. Note that CSV files that are not yet scanned live in a Docker volume (`tagscan_data`), not in the database; the copy in the Pi's 'sent' folder protects you meanwhile.
- **New Pi or new scanner.** Register it under Scanners, generate its own key, and repeat Part 8 with that key.
- **Lost, stolen or replaced Pi.** Immediately open Scanners > key icon > **Revoke key**. The device can no longer upload anything.
- **Changing a key** ('Generate new key') invalidates the old one instantly. Update `/etc/tagscan-watcher/prod.env` and run `sudo systemctl restart tagscan-watcher@prod` in the same sitting.

Troubleshooting table

Look for what you see, try the fix, and if it is still broken read the log mentioned in the last column.

Symptom	Most likely cause	Fix
ssh: connection refused / timed out	Wrong IP, VPS off, or firewall	Check the IP in hPanel; check ufw (Step 3.5) and hPanel firewall; check the VPS is running.
ssh: Permission denied (publickey)	Your public key is missing on the server	Redo Step 3.4c (correct file: /home/deploy/.ssh/authorized_keys, permissions 700 / 600).
certbot: 'Timeout during connect' / 'Challenge failed'	DNS not ready, or port 80 blocked	Steps 3.2 and 3.5; wait; run again.
Browser: 502 Bad Gateway	Container not running	docker compose -f docker-compose.prod.yml ps and logs backend, inside the environment folder.
Cannot log in to the site	Wrong address in .env (NEXT_PUBLIC_API_URL / CORS_ALLOWED_ORIGINS)	They must be exactly https://test.rwcrew.eu (or rwcrew.eu), no trailing slash; then up -d --build.
No TagScan tile after login	Module access not granted	Step 6.1 (Manage Access).
Watcher: 'Missing required environment variable'	Env file missing or misspelled	Check /etc/tagscan-watcher/dev.env has all four required lines; restart the service.
Watcher: HTTP 401	Wrong, revoked or other-environment key	Generate a new key on that site; update the env file; restart.
Watcher: HTTP 413	nginx limit missing (message contains nginx) or file over 20 MB	Step 3.8d; otherwise check file size.
Watcher: HTTP 503	Speed limit reached	Nothing; it retries next round.
Watcher: 'certificate verify failed'	Pi clock wrong	Step 8.2: sudo timedatectl set-ntp true.
Watcher: 'Connection refused / timed out'	No internet or DNS on the Pi	curl -I https://test.rwcrew.eu/api/health on the Pi.
Watcher: PermissionError (Errno 13)	Watcher user cannot use the folders	Repeat the chown / chmod of Step 8.5; folders must be under /var/lib/tagscan, not /home.
Files never leave 'outgoing'	Not named *.csv, still changing, or the service is stopped	Rename rule in Step 8.10; systemctl status tagscan-watcher@dev.
Your app cannot write into 'outgoing'	App user not in the group	Step 8.5 (usermod), then restart the application.
File in Unreaded Tags but Scan finds no lines	CSV column names differ	Appendix D: exact header names.
Upload URL in Settings shows localhost	NEXT_PUBLIC_API_URL wrong in .env	Fix .env on the VPS, rebuild (Step 6.4).

Appendix A. Command cheat sheet

Task	Command (where)
Log in to the VPS	ssh deploy@YOUR_VPS_IP (PC)
Log in to the Pi	ssh YOUR_PI_USER@YOUR_PI_HOST (PC)
Container status (in /opt/rwcrew/test or /production)	docker compose -f docker-compose.prod.yml ps (VPS)
Follow backend log	docker compose -f docker-compose.prod.yml logs -f backend (VPS)
Restart backend	docker compose -f docker-compose.prod.yml restart backend (VPS)
Redeploy by hand	bash deploy/scripts/deploy.sh /opt/rwcrew/test (VPS)
Check nginx settings and reload	sudo nginx -t && sudo systemctl reload nginx (VPS)
Renew test	sudo certbot renew --dry-run (VPS)
Watcher status	sudo systemctl status tagscan-watcher@prod (Pi)
Watcher live log	sudo journalctl -u tagscan-watcher@prod -f (Pi)
Restart watcher after config change	sudo systemctl restart tagscan-watcher@prod (Pi)
Stop / start watcher	sudo systemctl stop start tagscan-watcher@prod (Pi)
Show waiting files	ls /var/lib/tagscan/prod/outgoing (Pi)

Appendix B. Worksheet: values to write down

Fill this in as you go. Store the finished sheet in your password manager, not on paper lying around and never in Git.

Item	Your value	Where it comes from
YOUR_VPS_IP		Hostinger hPanel (Step 3.1)
VPS root password		You chose it (Step 3.1)
deploy user password		Step 3.4a
Test: database password		Step 4.2 (first random value)
Test: JWT secret		Step 4.2 (second random value)
Test: admin email + password		Step 4.4
Test: scanner API key		Step 6.3 (shown only once!)
Test: upload URL	https://test.rwcrew.eu/api/public/tagscan-intake	Step 6.4
Production: database password		Step 9.2
Production: JWT secret		Step 9.2
Production: admin email + password		Step 9.2
Production: scanner API key		Step 9.3
Production: upload URL	https://rwcrew.eu/api/public/tagscan-intake	Step 9.3
YOUR_PI_HOST / YOUR_PI_USER		Your Pi setup
YOUR_APP_USER (user of the Pi application)		Step 8.5

Appendix C. Final checklist

Server

- [] DNS: rwcrew.eu, www and test point to the VPS (nslookup).
- [] Login only with SSH key as user deploy; root and password logins disabled.
- [] Firewall on: OpenSSH, 80, 443.
- [] Docker works; nginx configured; certificates issued; renew dry-run succeeds.
- [] nginx file contains client_max_body_size 25m (twice) and the limit_req rule.
- [] Test site running at https://test.rwcrew.eu; admin can log in and has TagScan access.
- [] Production site running at https://rwcrew.eu.

TagScan and Pi

- [] Scanner registered; API key generated and stored safely; upload URL copied (test and production).
- [] curl upload from the PC worked: 201 received, then duplicate; wrong key gives 401; 4 MB file accepted.
- [] Pi clock correct; Pi can open https://.../api/health.
- [] Watcher files installed; env file (mode 600) filled in; service enabled and running.
- [] Test file from the Pi arrived in Unreaded Tags; Scan loaded it.
- [] Pi application writes .csv.tmp then renames to .csv, with unique names, into the outgoing folder.
- [] Production watcher running with the production key; test watcher switched off.
- [] Backups configured (database nightly, VPS snapshots).

Appendix D. The CSV file format TagScan expects

Files must be plain text CSV (comma separated, UTF-8; a BOM at the start is fine), one header line followed by one line per tag reading. Column names are matched by **name**, exactly as written, in any order.

Column	Meaning	Notes
Scanner	Name of the scanner	Should match the name in the Scanners screen; the first non-empty value in the file is used for the file.
EPC	The tag's electronic product code	Rows with an empty EPC are skipped.
RSSI (raw)	Signal strength	Whole number; unreadable values are stored as empty.
Antenna	Antenna number	Whole number.
Count	How often the tag was seen	Whole number.
Last Seen	Date and time of the last reading	Kept as text exactly as written.

Example:

FILE CONTENT
Scanner,EPC,RSSI (raw),Antenna,Count,Last Seen Scan-01,E2801160600002054F6A0001,-55,1,3,2026-09-20 10:15:00 Scan-01,E2801160600002054F6A0002,-61,2,1,2026-09-20 10:15:02

File names must end in **.csv**, be unique, and stay below 20 MB. Upload limits: 25 MB at nginx, 20 MB in the app.